

Tests Backend — Night Sky Viewer

Lancer les tests

```
# Depuis la racine du projet

# Activer le venv du backend (si nécessaire)
# Windows :
backend\venv\Scripts\activate
# Linux/Mac :
source backend/venv/bin/activate

# Lancer TOUS les tests backend
python -m pytest -v

# Lancer un fichier de test spécifique
python -m pytest tests/backend/test_observation_points.py -v
python -m pytest tests/backend/test_constellations.py -v

# Lancer avec couverture de code
python -m pytest --cov=backend/app --cov-report=term-missing

# Lancer une classe de test précise
python -m pytest tests/backend/test_observation_points.py::TestKnownObservationPoints -v

# Lancer un test unique
python -m pytest
tests/backend/test_constellations.py::TestKnownConstellations::test_orion_exists -v
```

Inventaire des tests existants

tests/backend/test_observation_points.py (~19 tests)

Classe	Tests	Description
TestObservationPointCount	1	Vérifie qu'il y a exactement 50 points d'observation en BD
TestObservationPointDataQuality	5	Noms non vides, latitudes [-90,90], longitudes [-180,180], timezone, unicité noms
TestKnownObservationPoints	5	Paris, Tokyo, New York, Mauna Kea, Atacama existent avec coordonnées correctes
TestObservationPointsAPI	8	GET /api/observation-points : status 200, retourne liste de 50, champs requis, tri alpha, GET /{id}, 404

tests/backend/test_constellations.py (~12 tests)

Classe	Tests	Description
--------	-------	-------------

TestConstellationCount	1	Vérifie 88 constellations IAU en BD
TestConstellationData	4	Abréviations 2-3 lettres majuscules, noms non vides, JSON lines_data valide, coordonnées centrales
TestConstellationStarAssociations	3	Associations >500, FK vers étoiles valides, FK vers constellations valides
TestKnownConstellations	4+	Orion (ORI) + Betelgeuse, Ursa Major (UMA), Cassiopeia (CAS), Crux (CRU), abréviations uniques

Tests à implémenter

tests/test_stars.py (priorité haute)

Données BD :

- Vérifier > 100 000 étoiles importées
- Qualité : RA ∈ [0, 360], Dec ∈ [-90, 90], magnitude non null
- Étoiles connues : Sirius (HIP 32349), Vega (HIP 91262), Polaris (HIP 11767)
- Unicité des `hip_id` (sauf null)

API :

- GET `/api/stars/visible?lat=48.86&lon=2.35` → status 200, retourne une liste
- GET `/api/stars/visible` sans params → status 422
- GET `/api/stars/1` → status 200, champs requis (id, ra, dec, magnitude)
- GET `/api/stars/999999` → status 404
- GET `/api/stars/search/Sirius` → status 200, résultats non vides
- GET `/api/stars/search/X` → status 400 (< 2 caractères)

tests/test_services.py (priorité haute)

AstronomyService :

- `compute_single_position` : Polaris depuis Paris → altitude ≈ 48° (±5°)
- `compute_single_position` : étoile sous l'horizon → `visible=False`
- `compute_visible_stars` : liste vide → retourne `[]`
- `compute_visible_stars` : retourne uniquement des étoiles avec altitude > 0

CacheService :

- `set` + `get` : stocker puis récupérer une valeur
- `get` sur clé inexistante → `None`
- `set_static` + `get_static` : même chose pour le cache statique
- `make_time_key` : deux timestamps proches (< 5 min) → même clé
- `make_time_key` : deux timestamps éloignés (> 5 min) → clés différentes
- `clear` : vide les deux caches

tests/test_api_constellations.py (priorité moyenne)

- GET `/api/constellations` → status 200, retourne 88 éléments
- GET `/api/constellations/1` → status 200, contient name, abbreviation, lines_data
- GET `/api/constellations/99999` → status 404

- `GET /api/constellations/search?q=Ori` → status 200, résultats non vides
- `GET /api/constellations/{id}/best-location` → status 200, contient `observation_point_id`

tests/test_rovers.py (priorité haute)

API :

- `GET /api/rovers/positions` → status 200, retourne une liste non vide
- Réponse contient les 5 rovers par défaut (`curiosity` , `perseverance` , `opportunity` , `spirit` , `zhurong`)
- Chaque rover a `slug` , `name` , `latitude` , `longitude` , `active` , `landing_site`
- `latitude` ∈ [-90, 90], `longitude` ∈ [-180, 180]
- Cohérence : Curiosity et Perseverance `active=true` , les 3 autres `active=false`

tests/test_satellites.py (priorité moyenne)

Service :

- `parse_tle_text` : texte TLE valide (3 lignes) → retourne 1 `SatelliteTLE`
- `parse_tle_text` : texte vide → retourne `[]`
- `parse_tle_text` : lignes malformées (longueur < 69) → ignorées

API :

- `GET /api/satellites/tle` (sans params) → status 200, groupe `stations` par défaut
- `GET /api/satellites/tle?group=gps-ops` → status 200, `count` > 0
- `GET /api/satellites/tle?group=inconnu` → status 400
- `GET /api/satellites/groups` → status 200, liste de 8 groupes
- Réponse contient `group` , `count` , `satellites`
- Chaque satellite a `name` , `line1` , `line2` ; `line1` commence par "1 " , `line2` par "2 "

tests/test_health.py (priorité basse)

- `GET /` → status 200, contient `status: "running"`
- `GET /api/health` → status 200, contient `database` , `star_count`

Tests Frontend (React / Vitest)

Les tests frontend sont situés dans `tests/frontend/` . La configuration est faite via `vitest.config.ts` (à la racine) et utilise `@testing-library/react` combiné à `jsdom` .

```
# Depuis la racine du projet
# Installez les dépendances de test à la racine si ce n'est pas déjà fait :
npm install -D vitest @testing-library/react @testing-library/jest-dom jsdom @vitest/browser

# Lancer la suite de test frontend
npx vitest run

# Mode interactif watch
npx vitest
```